

ICS EXE 4

2026.03.27

Problem1: Cache

Consider a **12-bit machine** with a **2-way set** associative cache. The size of each block is **4 bytes**, and the total number of sets is 4. **LRU** policy is used for eviction and **write back** policy is used. The following table shows the content of the data cache at time T.

Set	Tag	Valid	Byte0	Byte1	Byte2	Byte3	Tag	Valid	Byte0	Byte1	Byte2	Byte3
0	0x08	1	0xAC	0xAB	0x08	0x60	0x6A	1	0x11	0x1F	0x64	0x04
1	0xFE	1	0x56	0x01	0x00	0xCC	0x92	0	0xCA	0xBA	0x9F	0xF0
2	0x08	0	0x4C	0xAB	0xFF	0xFF	--	--	--	--	--	--
3	--	--	--	--	--	--	0x92	1	0x93	0x10	0xFD	0xBB

One of the TA write a program and test it on this cache. The size of short type is **2 bytes**. The cache is **empty** before the execution of the program. Please only consider **data cache** access and ignore instruction cache access. Other processes will not interfere status of memory and cache during this execution.

```
1. short data1[4][2];
2. short data2[2][4];
3.
4. void total() {
5.     int i, j;
6.
7.     for (i = 0; i < 2; i++) {
8.         for (j = 0; j < 4; j++) {
9.             data1[j][i] = data2[i][j] * i;
10.        }
11.    }
12. }
```

Assume the address of data1[0] is 0x00. The address of data2[0] is 0x40. Answer the following questions:

1. Please calculate the cache miss rate. (2')
2. Is there a way to optimize the cache miss rate if we **only** reorder some part of the code? (2')
3. We can modify the cache through one of the following ways:
 - a) Double the number of sets while keeping the cache size unchanged.
 - b) Double the block size while keeping the cache size unchanged.
 - c) Increase the cache size via doubling the number of cache lines in each set.

Please choose the option(s) above that can optimize the miss rate of the **original program** (2'), **and explain why** (2')

Problem 2: Cache

Jack has a **64-bit machine** with a **2-way set** associative cache. There are **4 sets**. Each block is **8 bytes**. The following table shows the content of the data cache at time T. **ByteX** is the byte value stored at offset **X**. Assume the cache uses **LRU** and **write back** policy.

Set	Tag	Valid	Dirty	Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0	--	0	--	--	--	--	--	--	--	--	--
	0x401b5	1	0	0x46	0xaf	0xfa	0x36	0x92	0x45	0x01	0x37
1	0x40136	1	1	0x46	0x4a	0x11	0x08	0x33	0x47	0x04	0xda
	--	0	--	--	--	--	--	--	--	--	--
2	0x401b5	0	0	0x42	0x08	0x07	0x66	0x62	0x6a	0x6f	0x9f
	0x8036b	1	0	0x23	0x2a	0x73	0x75	0xe7	0xef	0x2e	0x88
3	0x2009c	1	1	0x05	0x81	0xa2	0xa3	0x83	0x30	0x40	0x32
	0x40139	1	1	0xdd	0xda	0xcd	0xc5	0x6c	0xaf	0x9d	0x90

- How would a 64-bit physical machine memory address be split into tag/set-index/block-offset fields in this machine? ($2^3=6$)

tag [1] bits **set-index** [2] bits **block-offset** [3] bits

- Why caches use the **middle bits** for the set index instead of the **high-order bits**? (2')
- A short program will access memory in the following sequences starting from time T. Assume there are **no other memory accesses** from other programs or kernel. Each memory access will **read or write 1 byte**. Please fill in the following blanks. If the value does not exist or cannot be determined, fill the blank with '--'. ($1^*12=12$)

Order	Operation	Hit/Miss	Byte Returned
1	Read 0x401368	[1]	[2]
2	Read 0x8036a5	[3]	[4]
3	Write 0x8036b3	[5]	[6]
4	Write 0x4014aa	[7]	[8]
5	Write 0x40139b	[9]	[10]
6	Read 0x8026cf	[11]	[12]

4. Assume the program will **flush all data back to memory** in this cache after the memory accesses in question 3. How many cache lines will be written to memory **totally**? (Note: **Calculate from time T.**) (2')

Jack writes a program and test it on this cache. The size of **int** value is **4 bytes**. The cache is **empty** before each execution. Please only consider **data cache** access. Assume other processes do not modify the memory and cache during this execution.

```
1.  int a[8][2];
2.  int b[8][2];
3.
4.  int fun(void) {
5.      int i, j, sum = 0;
6.
7.      for (j = 0; j < 2; j++)
8.          for (i = 0; i < 8; i++)
9.              sum += a[i][j] * b[i][j];
10.
11.     return sum;
12. }
```

Assume the address of **a[0][0]** is **0x00**. The address of **b[0][0]** is **0x40**. Answer the following questions:

5. Please calculate the cache miss rate. (2')
6. How to optimize the cache miss rate if we only **reorder** some part of the code? (2')
7. Now Jack can enlarge the cache size through **one of** the following ways:
- A. Double the number of sets.
 - B. Double the number of cache lines in each set.
 - C. Double the block size of each cache line.

Please choose all options above that can optimize the miss rate of the **original program**. (2')